

习题讲解

Question 15

The Japanese game
grid. Players take to
markers consecutiv
game generalized to



GM =

By a *position* we mea
middle of a play of
next. Show that GM

要证明PSPACE，一般是要给出一个递归算法（对于叶子节点。。对于非叶子节点。。），分析这个递归算法的最大深度，分析求解的空间消耗（节点之外要存什么，每个节点存什么，一个路径最多有几个节点，节点这块的空间消耗 = 每个节点*节点数量）

采用空间复用技术，所以总消耗 = 最大分支的消耗

有必胜策略：B这个位置，X已经有一个必胜策略了

递归地考虑：

从B这个position出发

对叶子节点，可以直接判断是否X赢，返回结果给父节点

对中间节点

如果是X落子，尝试X的一种合法的、之前没有尝试过的落子方式，然后继续递归，这个节点为true当且

仅当所有子节点为true

如果是Y落子，尝试Y的一种合法的、之前没有尝试过的落子方式，然后继续递归，这个节点为true当且

仅当存在子节点为true

采用空间复用策略，当一个子分支得到结果就释放对应的空间，空间的最大消耗为某一条分支的最大消耗

空间消耗分析：

✓ 记录一整张棋盘的当前position，空间最大不超过 n^2 这个常数

记录当前是谁落子，空间消耗常数

每个节点：记录当前节点的落子位置

✓ 最大节点数 = 一个分支的最大节点数 $< n^2$

！总空间 = 最大递归深度 * 单层空间消耗 = $O(n^4)$

所以是PSPACE

19 × 19

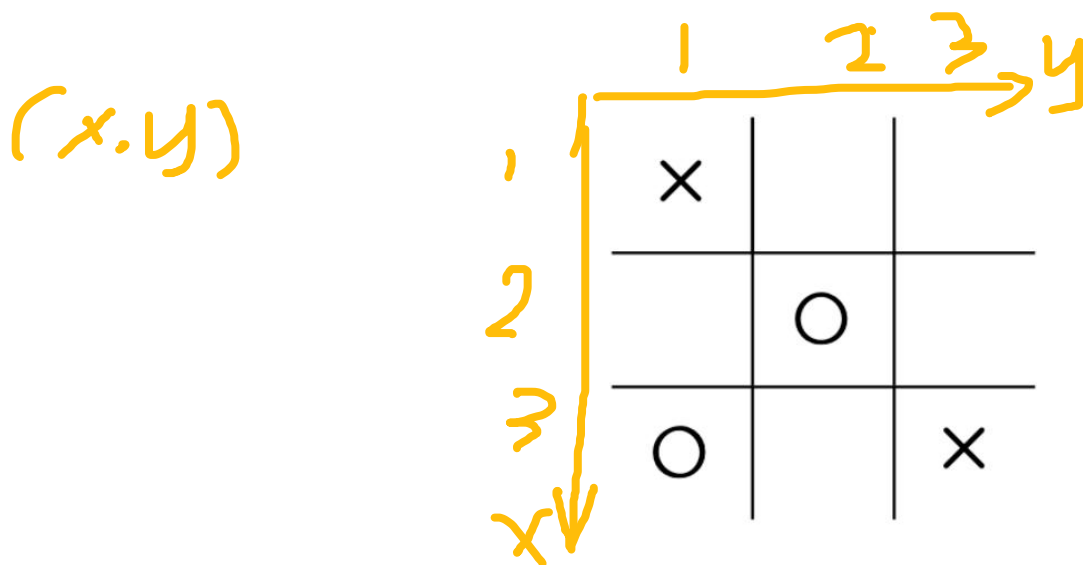
ve of her
sider this

ur in the
er moves

习题讲解

Question 16

Consider the following position in the standard tic-tac-toe game.



这个题和一般的证明题不一样，而是要你实际地给出一个下棋的策略
落子落在 (1, 3) 的位置，那么无论O下到那里，X都可以再下到 (1, 2) 或者 (2, 3) 取胜

Let's say that it is the $\times$ -player's turn to move next. Describe a winning strategy for this player. (Recall that a winning strategy isn't merely the best move to make in the current position. It also includes all the responses that this player must make in order to win, however the opponent moves.)

习题讲解

Let A be the language of properly nested parentheses. For example, $(())$ and $(()())$ are in A , but $) ($ is not. Show that A is in L .

一般L或者NL的问题，都是只能用二进制存一个或多个常数（具体可能是常数值，指针等）

一般和L或者NL相关的问题都需要区分input tape和working tape

针头从input tape从左向右移动

每次遇到“ (”号，working tape的针头就右移一位

反之就左移一位

如果working tape的针头在0处还要左移，那就直接停机，返回reject

如果遍历输入纸带结束，working tape的针头在0处，返回accept

其他情况均返回reject

空间消耗分析：working tape的针头，对应的数值用二进制表示，最大数值为输入的长度 n ，于是消耗空间最大是 $\log_2\{n\}$

习题讲解

二分图等价于，若2-color着色后，没有一条边的两个端点颜色相同
不是二分图等价于，从某个点开始，进行2-color着色后，存在一条边的两个端点颜色相同

An undirected graph is *bipartite* if its nodes may be divided into two sets so that all edges go from a node in one set to a node in the other set. Show that a graph is bipartite if and only if it doesn't contain a cycle that has an odd number of nodes. Let $BIPARTITE = \{\langle G \rangle \mid G \text{ is a bipartite graph}\}$. Show that $BIPARTITE \in NL$.

【这个还是听一下老师怎么讲吧。。】

注意题目有两个小问！

(1) 证明“等价性”(iff)，一般都是通过分别证明“充分性”(sufficiency)和“必要性”(necessity)来实现【反证法是个好东西！】

题目：是二分图 iff 无奇数环（含有奇数个节点的那种）

充分性：假设是二分图而且有奇数环，不妨设那个环上的节点按照连接顺序依次记为 $A_1, A_2, \dots, A_{2n+1}$ ，不妨设 A_1 在set1中。那么显然 A_2 在set2中，那么最终所有索引是奇数的点都在set1中，所以 A_{2n+1} 在set1中，但是 A_1 和 A_{2n+1} 之间有边，这就和定义相违背

必要性：无奇数环 $\Rightarrow$ 是二分图，反正法是对结论取反！！反证法：已知无奇数环 $\Rightarrow$ 不是二分图（等价性质见上框）

官方证明：染色法；从某一个节点开始，给他染白色，给相邻的节点染黑色，如此交替不同颜色，直到所有节点都染上一种颜色

如果不存在同一边的两个节点颜色相同，那么显然这就是二分图

反之，如果存在，记这条边两端的节点是 u 和 v ，那么从 v_0 到 u 和从 v_0 到 u 的路径加起来显然是偶数，那么 v_0 和 u ， v 构成的环的长度就是“偶数+奇数”就是奇数，所以这和假设不成立，所以不存在同一边的两个节点颜色相同【二分图】

NL就是认为如果存在一个解，那么我们就可以直接找到这个解，于是空间消耗就是这一次查找的消耗，正向难找，可以试试找补集

(2) NL定义：只能存指针，但是可以准确猜出下一个位置（非确定性地选择起始位置和下一个位置）

证明补集：【不是二分图】

算法如下：

非确定性地选择一个起始位置

维护当前指针 cur ，维护当前的步数计数器 $count$ （初始为0）

非确定地选择一个邻居，更新 $cur=next$ ， $count++$

如果最终回到起点（ $next = \text{起始位置}$ ），并且 $count = \text{奇数}$ ，那么就accept

空间分析（要存储什么东西）：初始节点， cur 和 $count$ （相当于每个节点都有一个独立的索引值，并且最大是 n ，而 $count$ 最大是总边数量，

最大是 n ）

习题讲解

误区！两条边之间有有向路径，并不是有直接的一条边！而是“可达”，对应的就是PATH问题

误区！PATH问题是特定的两个点之间可达

误区！从PATH进行多项式规约，初始的图是PATH的图（这图只知道特定的两个点可达），通过调整变成了G的图

Recall that a directed graph is *strongly connected* if every two nodes are connected by a directed path in each direction. Let

$$STRONGLY-CONNECTED = \{ \langle G \rangle \mid G \text{ is a strongly connected graph} \}.$$

Show that *STRONGLY-CONNECTED* is NL-complete.

凡是X-complete问题，基本都是要分别证明是X和X-hard

证明是NL；

✓ 反证法：证明不是强连通图，等价于证明存在两个点uv，从u到v没有一条有向路径

NL：如果有这样的uv，能直接猜出来，并且验证不存在有向路径，于是在这样的条件下问题变成了PATH问题，而我们已知PATH问题可以在多项式空间内解决，所以强连通图的补集是NL问题，又因为NL对补集closed，所以得证

证明是NL-hard：（即证明所有的NL问题都可以规约到这个问题）通过证明NL-hard问题可以规约到它来证明

注意！L/NL的规约采用的是多项式空间规约！给出一个方法，将A的E构建到B的E'，要求：1、构建的过程使用的空间是多项式级别的2、符合规约的定义：构造后：E属于A等价于E'属于B

本题从PATH问题规约：PATH问题的图G，构造成strongly-connected的G'

从s到t有路径 等价于 G' 是强连通图

！！构造方法：增加点和边：

增加t到所有节点的边，增加所有节点到s的边，就可以实现要求

空间使用分析【只关心转换器的空间需求，而不关心输入输出的空间需求】：只需要遍历所有的节点并输出边，所以只需要维护当前的索引，消耗的空间是O(logn)

习题讲解

Prove that $\text{TIME}(2^n) = \text{TIME}(2^{n+1})$.

证明集合相等 $A = B$ 的一般方法是，分别证明 A 包含于 B 和反过来
 $\text{Time}(f(n))$ 的定义是，可以在 $O(f(n))$ 时间内解决的问题的集合

显然，能在 2^n 时间内解决的问题也可以在 2^{n+1} 时间内解决

关键在于利用大 O 符号忽略常数因子的特点

$$O(2^{n+1}) = O(2^n)$$

所以，能在 2^{n+1} 时间内解决的问题也可以在 2^n 时间内解决

期末考试复习

1. SAT的定义以及证明应用
2. CNF 的子句求值解析
3. 不同概念之间的关系: $P, PSPACE, NPSPACE, NP, NPC, NL, coNL$,
4. 图灵机模型: 多带图灵机的运行机制, 如何分析图灵机的运行时间上限
5. NP 问题证明
6. PSPACE 问题证明
7. undirected graph 性质, 定义以及相关证明
8. NPC , PSPACE-completeness

最好教授上课的推导都熟悉一下

SAT定义, 作业题里的例子, 证明SAT的小问题, 基本思路

CNF, 作业里有个取值关系的题目! 给出一个CNF公式, 判断在什么情况下是可以满足的

考试整体不会难, 关注上课的定义, 讲过的推理, 作业题的思路

注意写证明的时候不要跳步! 踩点给分, 简单的东西也要往上写! 写详细 (让老师知道你知道)

不同图灵机的运行机制, 概念的熟悉, 知道定义的核心要点

一定保证作业题要会 (70%以上), 太难的作业题可以放一放, 上面这些题都不算特别难

上90: 把上课讲的逻辑, 推理再熟悉一下

1:17